Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen 1 — Historia de la Programación y Fundamentos Universales de Informática

Partes I y II: De Babbage, Turing y Church a la Arquitectura de Computadores, Sistemas de Tipos, Paradigmas y Matemática Discreta

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I PARTE I. Historia de la Informática y la Programación 1

1. Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda 2
1.1. De Babbage y Ada Lovelace al álgebra de Boole 2
1.2. Alan Turing, Alonzo Church y los límites de lo computable 3

2. Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix 6
2.1. Código máquina, ensambladores, Grace Hopper y la primera generación de compiladores . 6
2.2. FORTRAN, Lisp, COBOL, ALGOL, programación estructurada, C y Unix 7

3. De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA 10
3.1. Smalltalk, C++, Java, Python, JavaScript, C# y el movimiento del software libre 10
3.2. Git, paralelismo multinúcleo, Go, Rust, Kotlin, Swift, Cloud, WebAssembly e IA 11

II PARTE II. Fundamentos Universales de Programación y Matemáticas Discretas 14

4. Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8 15
4.1. Bits, bytes, endianness, enteros sin signo y complemento a dos 15
4.2. Aritmética IEEE 754 y codificación de longitud variable UTF-8 15

5. Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos 19
5.1. Álgebra booleana, lógica formal e inducción matemática y estructural 19
5.2. Conjuntos, relaciones, funciones, combinatoria y grafos en computación 19

6. Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT 23
6.1. CPU, registros, pipeline, memoria principal, pila y montículo (stack y heap) 23
6.2. De código fuente a ejecución: interpretación, bytecode, compilación AOT y JIT 24

7. Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores 27
7.1. Taxonomía rigurosa de sistemas de tipos: estático, dinámico, nominal, estructural y algebraico 27
7.2. Ámbito léxico, duración, clausuras, acoplamiento modular y gestión de errores 28

8. Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección 31
8.1. Imperativo, procedural, orientado a objetos, declarativo, funcional y lógico 31
8.2. Genérico, eventos, reactivo, concurrente, asíncrono y programación de sistemas 32

A. Notación 35

B. Hoja de fórmulas **36**

C. Soluciones **37**
C.1. Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda 37
C.2. Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix 38
C.3. De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA 40
C.4. Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8 . 41
C.5. Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos 43
C.6. Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT 45
C.7. Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores 46
C.8. Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección 48

D. Auditoría de fuentes **50**

E. Plan de estudio **52**

Parte I

PARTE I. Historia de la Informática y la Programación

Capítulo 1

Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

Comprender por qué nacieron la ciencia de la computación y los lenguajes formales antes incluso de existir los ordenadores electrónicos: qué problema matemático motivó a Babbage, Lovelace, Boole, Church y Turing, y cómo la noción de programa almacenado e indecidibilidad condiciona toda herramienta moderna de ingeniería del software.

Consejo

Distinga siempre entre hechos históricos documentados y simplificaciones divulgativas: la arquitectura del informe EDVAC (1945) firmada por von Neumann sintetizaba el trabajo conjunto con Eckert y Mauchly en ENIAC/EDVAC; asimismo, el teorema de indecidibilidad no impide verificar programas concretos, sino que obliga a diseñar sistemas de tipos decidibles con restricciones sintácticas explícitas.

Definición

Tesis de Church-Turing y Teorema de Rice: toda función efectivamente calculable por un método algorítmico finito es computable por una Máquina de Turing (y equivalentemente expresable en cálculo lambda). Por el teorema de Rice, toda propiedad semántica no trivial del comportamiento de un programa en un lenguaje Turing-completo es indecidible en el caso general; por ello los sistemas de tipos y verificadores como el borrow checker de Rust adoptan análisis conservadores (rechazan algunos programas válidos para garantizar que ningún programa aceptado viole la propiedad de seguridad).

Autoficha

1. ¿Por qué la Máquina Universal de Turing implica que el código y los datos comparten naturaleza formal? 2. ¿Qué diferencia fundamental separa la raíz imperativa (Turing/von Neumann, mutación de estado en memoria) de la raíz funcional (Church, reducción de expresiones lambda sin estado mutable)? 3. ¿Por qué el teorema de Rice exige que un analizador estático seguro sea conservador?

1.1 De Babbage y Ada Lovelace al álgebra de Boole

La historia de la programación no es un catálogo cronológico de sintaxis, sino la respuesta progresiva a una restricción fundamental: cómo expresar procesos mecánicos de deducción y cálculo sin ambigüedad humana. En el siglo XVII, las máquinas aritméticas de Wilhelm Schickard, Blaise Pascal y Gottfried Wilhelm Leibniz mecanizaron la propagación del acarreo, pero carecían de bifurcación condicional y de memoria direccionable. Cuando Charles Babbage diseñó la Máquina Analítica (1837), separó por primera vez el almacén de datos (Store, precursor de la memoria principal) del molino de procesamiento (Mill, precursor de la unidad aritmético-lógica), gobernados mediante tarjetas perforadas inspiradas en el telar de Jacquard. En 1843, Augusta Ada King, condesa de Lovelace, tradujo y amplió la memoria de Luigi Menabrea con sus célebres Notas A a G. En la Nota G formuló el primer algoritmo publicado destinado a ejecutarse en una máquina con ramificación e iteración: el cálculo de los números de Bernoulli. Más importante aún fue su intuición conceptual: advirtió que la máquina podía manipular cualquier entidad simbólica cuyas relaciones internas pudieran expresarse mediante reglas operacionales, anticipando la distinción moderna entre representación física y semántica de tipos. Una década más tarde, George Boole publicó An Investigation of the Laws of Thought (1854), reduciendo la lógica proposicional a un álgebra binaria sobre $\{0, 1\}$ que Claude Shannon conectaría en 1937 con los circuitos de conmutación.

1.2 Alan Turing, Alonzo Church y los límites de lo computable

El nacimiento riguroso de la informática teórica se produjo en 1936 a raíz del Entscheidungsproblem (problema de la decisión) planteado por David Hilbert: ¿existe un procedimiento mecánico finito capaz de decidir la validez de cualquier fórmula lógica de primer orden? Alonzo Church respondió negativamente mediante el cálculo lambda, un sistema formal basado exclusivamente en abstracción de funciones y aplicación por sustitución que constituye la raíz directa de Lisp, ML, Haskell, OCaml y las clausuras léxicas de todos los lenguajes actuales. Simultáneamente, Alan M. Turing formalizó la noción de calculador mecánico mediante una máquina abstracta de estados finitos acoplada a una cinta ilimitada de símbolos, como ilustra la figura 1. La aportación decisiva de Turing fue triple. Primero, definió con precisión matemática qué significa que una función sea computable. Segundo, construyó la Máquina Universal de Turing (UTM), capaz de leer en su cinta la descripción codificada de cualquier otra máquina y simular su ejecución: los programas pasaron a ser datos, principio que John von Neumann, J. Presper Eckert y John Mauchly materializarían en el informe del EDVAC (1945). Tercero, demostró mediante diagonalización la indecidibilidad del problema de la parada (Halting Problem): ningún analizador estático ni compilador puede decidir de forma exacta y universal para todo programa arbitrario si terminará o si estará libre de toda propiedad semántica no trivial (teorema de Rice). Podemos verificar constructivamente en Python un simulador determinista de máquina de Turing de tres estados:

```
def run_turing_increment(bits: list[int]) -> tuple[list[int], int]:
    tape = list(bits)
    head = len(tape) - 1
    steps = 0
    state = 'CARRY'
    while state != 'HALT':
        steps += 1
        if head < 0:
            tape.insert(0, 1)
            state = 'HALT'
        elif tape[head] == 1:
            tape[head] = 0
            head -= 1
        else:
            tape[head] = 1
            state = 'HALT'
    return tape, steps

out, n_steps = run_turing_increment([1, 0, 1, 1])
assert out == [1, 1, 0, 0] and n_steps == 3
```

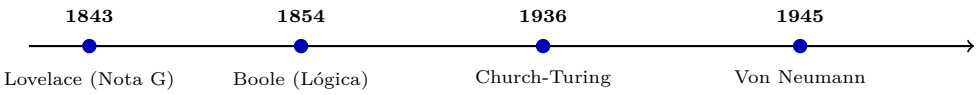


Figura 1. Línea evolutiva desde el cálculo mecánico y la lógica booleana hasta la máquina universal de Turing y la arquitectura de programa almacenado.

Problema resuelto

Enunciado Considere una Máquina de Turing determinista con $|Q| = 6$ estados activos, un alfabeto de cinta de $|\Gamma| = 4$ símbolos y $d = 2$ direcciones de movimiento del cabezal. Determine el número total de entradas de la función de transición $\delta : Q \times \Gamma \to (Q \cup \{\text{HALT}\}) \times \Gamma \times \{L, R\}$ y el número de bits necesarios por entrada si cada transición codifica uno de los $6 + 1 = 7$ estados destino (3 bits), el símbolo escrito (2 bits) y la dirección (1 bit), así como el tamaño total en bits de la tabla de transición.

Datos

- $Q = 6$ — Número de estados activos
- $G = 4$ — Cardinalidad del alfabeto de cinta

- $b_{entry} = 6$ — Bits por transición (3 + 2 + 1)

Incógnita Tamaño total en bits de la tabla de transición codificada
Modelo físico
Codificación finita de la tabla de transición de una Máquina de Turing determinista

$$B_total = N_pairs * b_entry$$

Hipótesis

- Cada par (estado, símbolo leído) posee exactamente una transición definida
- Codificación de ancho fijo de 6 bits por entrada de transición

Desarrollo

1. Calculamos el número de pares (estado activo, símbolo leído) en el dominio de la función de transición.

$$N_pairs = Q * G$$

$$(6 \cdot 4)$$

2. Multiplicamos el número de entradas por los 6 bits requeridos para codificar estado destino, símbolo y dirección.

$$B_total = N_pairs * b_entry$$

$$(24 \cdot 6)$$

Resultado

144

Interpretación La máquina completa queda descrita de manera unívoca por 144 bits (18 octetos), lo que ilustra cómo una Máquina Universal puede almacenar la descripción completa de un autómata como dato finito.
Comprobación de los pasos

- Paso 1: 24 — reproducido
- Paso 2: 144 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- La codificación de longitud fija desaprovecha estados si el número de estados no es potencia de dos.

Errores frecuentes

- Olvidar incluir el estado de parada HALT dentro del conjunto de estados destino codificables.

Ejercicios

1 [PRB-0003] (Básico, Conceptual) [VOL01-C01-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la

expresión reproducible `12 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0004] (Básico, Numérico) [VOL01-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0005] (Intermedio, Interpretación) [VOL01-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0006] (Intermedio, Aplicación) [VOL01-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(12 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0007] (Intermedio, Identificación de errores) [VOL01-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0008] (Avanzado, Desarrollo por pasos) [VOL01-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0009] (Avanzado, Comparación) [VOL01-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0010] (Avanzado, Comprobación de hipótesis) [VOL01-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0011] (Nivel de examen, Optimización) [VOL01-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0012] (Nivel de examen, Demostración) [VOL01-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0013] (Reto, Diseño) [VOL01-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0014] (Reto, Estudio de parámetros) [VOL01-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 2

Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

Explicar por qué cada generación de lenguajes entre 1945 y 1980 surgió como respuesta directa a una crisis de ingeniería: el coste prohibitivo de codificar en binario, el escepticismo frente al código generado por compiladores, el caos del flujo con saltos arbitrarios (`GOTO`) y la necesidad de escribir sistemas operativos portables.

Consejo

Comprenda el contexto tecnológico de cada decisión de diseño: las cadenas terminadas en byte nulo (`'\0'`) de C se eligieron en el PDP-11 para ahorrar el prefijo de longitud en memorias de pocos kilobytes, pero introdujeron el coste $O(n)$ para calcular la longitud y el riesgo histórico de desbordamiento de búfer; por eso Rust, Go y Swift representan hoy las vistas de cadena como pares `(puntero, longitud)` (`&str`, `string`, `String`).

Definición

Teorema del Programa Estructurado (Böhm-Jacopini, 1966) y Gramática BNF: toda función computable representable mediante un grafo de flujo de control arbitrario puede expresarse combinando únicamente tres estructuras de control: secuencia, selección condicional (`if-else`) e iteración controlada por predicado (`while`). La notación Backus-Naur Form (BNF), introducida en ALGOL 60, define formalmente la sintaxis de un lenguaje mediante reglas de producción sobre símbolos terminales y no terminales.

Autoficha

1. ¿Por qué el compilador FORTRAN I tuvo que priorizar la optimización agresiva de bucles para ser aceptado? 2. ¿Qué innovaciones fundamentales aportaron Lisp (1958) y ALGOL 60 (1960) a los lenguajes modernos? 3. ¿Qué relación existe entre la disciplina de programación estructurada de Dijkstra y la verificación por invariantes?

2.1 Código máquina, ensambladores, Grace Hopper y la primera generación de compiladores

En los primeros ordenadores electrónicos de finales de los años cuarenta (ENIAC, Manchester Baby, EDSAC de Maurice Wilkes), programar consistía en cablear paneles o introducir palabras binarias y octales directamente en memoria. Cualquier inserción de una instrucción desplazaba las direcciones absolutas de todas las subrutinas posteriores. Los primeros ensambladores simbólicos (como las Initial Orders de David Wheeler en EDSAC, 1949) introdujeron mnemónicos y etiquetas relocalizables, pero cada familia de procesadores seguía atada a un dialecto incompatible. En 1952, Grace Murray Hopper desarrolló el sistema A-0 para el UNIVAC I, acuñando el término compiler para describir un programa que ensamblaba subrutinas a partir de especificaciones simbólicas, y posteriormente impulsó FLOW-MATIC, demostrando que un lenguaje podía aproximarse a notación legible para procesamiento de datos empresariales. En paralelo, John Backus dirigió en IBM el diseño de FORTRAN (1957) para el IBM 704. En aquella época los programadores de código máquina desconfiaban de cualquier traductor automático por temor a perder ciclos de CPU; por ello el compilador FORTRAN I incorporó análisis de flujo e inferencia de registros en

bucles `DO`, logrando código objeto cuya eficiencia rivalizaba con el ensamblador manual y reduciendo por un orden de magnitud el tiempo de desarrollo.

2.2 FORTRAN, Lisp, COBOL, ALGOL, programación estructurada, C y Unix

Entre 1958 y 1960 cristalizaron cuatro linajes que aún vertebran la informática moderna. John McCarthy creó Lisp (1958) en el MIT para manipulación simbólica e inteligencia artificial, introduciendo listas enlazadas, expresiones condicionales, funciones de orden superior, homoiconicidad (el código es una estructura de datos del propio lenguaje) y recolección automática de basura (Garbage Collection). El comité CODASYL, con fuerte influencia de los diseños de Grace Hopper, definió COBOL (1959) con registros jerárquicos y aritmética decimal exacta para sistemas financieros. En 1960, un comité internacional publicó el informe de ALGOL 60, que introdujo el ámbito léxico por bloques, la recursividad general, el paso por valor y por nombre, y la notación formal BNF (Backus-Naur Form) para especificar gramáticas libres de contexto. En 1968, Edsger W. Dijkstra publicó su carta Go To Statement Considered Harmful y, junto con Böhm, Jacopini, Hoare y Wirth (creador de Pascal en 1970), consolidó la programación estructurada: todo flujo secuencial, condicional e iterativo puede componerse con bloques de una única entrada y una única salida, haciendo posible razonar mediante invariantes de Hoare. Finalmente, en los Laboratorios Bell, Ken Thompson y Dennis Ritchie desarrollaron Unix y el lenguaje $C (1972 - 1973)$. Al reescribir el núcleo de Unix en C para el PDP-11, demostraron que un lenguaje con aritmética tipada de punteros y mapeo directo al hardware permitía portar un sistema operativo completo entre arquitecturas distintas recompilando el código fuente:

```
#include <assert.h>
#include <stddef.h>

size_t c_str_len(const char *s) {
    const char *p = s;
    while (*p != '\0') {
        p++;
    }
    return (size_t)(p - s);
}

int main(void) {
    assert(c_str_len("ALGOL-60") == 8);
    return 0;
}
```

Problema resuelto

Enunciado En un computador histórico con memoria de $M = 32768$ palabras de 36 bits, comparar el espacio ocupado por una tabla de $N = 1024$ identificadores de 8 caracteres ASCII (8 bits por carácter) cuando se almacenan como cadenas con centinela nulo de 1 byte frente a cadenas con prefijo de longitud de 4 bytes sin centinela. Calcule el ahorro neto en bytes de la representación con centinela de 1 byte para los $N = 1024$ identificadores.

Datos

- $N = 1024$ — Número de cadenas almacenadas
- $L_{null} = 9$ — Bytes por cadena con centinela nulo $(8 + 1)$
- $L_{pref} = 12$ — Bytes por cadena con prefijo de 4 bytes $(8 + 4)$

Incógnita Ahorro en bytes de la representación con centinela nulo de 1 byte

Modelo físico

Comparación de sobrecarga de metadatos por cadena en memoria lineal

$$S_bytes = N * delta_b$$

Hipótesis

- Cada carácter ocupa exactamente 1 octeto y no se añade relleno adicional de alineamiento

Desarrollo

1. Determinamos la diferencia de bytes por cadena entre el prefijo de 4 bytes y el centinela de 1 byte.

$$delta_b = (8 + 4) - (8 + 1)$$

$$((8 + 4) - (8 + 1))$$

2. Multiplicamos el ahorro unitario por las 1024 cadenas de la tabla.

$$S_bytes = N * delta_b$$

$$(1024 \cdot 3)$$

Resultado

$$3072$$

Interpretación En memorias de pocas decenas de kilobytes, ahorrar 3072 bytes en metadatos justificó la adopción de cadenas terminadas en nulo en C; en sistemas modernos con gigabytes de RAM, el coste O(n) de `strlen` y los errores de seguridad hacen preferible almacenar `(ptr, len)`.
Comprobación de los pasos

- Paso 1: 3 — reproducido
- Paso 2: 3072 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Si la longitud máxima es menor de 255 bytes, un prefijo de 1 byte (estilo Pascal) iguala el coste espacial con consulta O(1).

Errores frecuentes

- Ignorar que las cadenas con centinela nulo no pueden contener el octeto `0x00` como dato válido.

Ejercicios

1 [PRB-0017] (Básico, Conceptual) [VOL01-C02-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0018] (Básico, Numérico) [VOL01-C02-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0019] (Intermedio, Interpretación) [VOL01-C02-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 3 + 4` y justifique los criterios de aceptación,

pistas graduadas y complejidad temporal/espacial.
4 [**PRB-0020**] **(Intermedio, Aplicación)** [VOL01-C02-E04] (Nivel 2— Aplicación (Predicción determinista de salida)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(15 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [**PRB-0021**] **(Intermedio, Identificación de errores)** [VOL01-C02-E05] (Nivel 3— Resolución (Detección y corrección de defecto sutil)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [**PRB-0022**] **(Avanzado, Desarrollo por pasos)** [VOL01-C02-E06] (Nivel 3— Resolución (Implementación multi-etapa con invariante)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [**PRB-0023**] **(Avanzado, Comparación)** [VOL01-C02-E07] (Nivel 4— Ingeniería (Refactorización modular y contratos)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [**PRB-0024**] **(Avanzado, Comprobación de hipótesis)** [VOL01-C02-E08] (Nivel 4— Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [**PRB-0025**] **(Nivel de examen, Optimización)** [VOL01-C02-E09] (Nivel 5— Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [**PRB-0026**] **(Nivel de examen, Demostración)** [VOL01-C02-E10] (Nivel 5— Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [**PRB-0027**] **(Reto, Diseño)** [VOL01-C02-E11] (Nivel 6— Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [**PRB-0028**] **(Reto, Estudio de parámetros)** [VOL01-C02-E12] (Nivel 6— Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 3

De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

Analizar las fuerzas tecnológicas que impulsaron los lenguajes y plataformas desde 1980 hasta la actualidad: interfaces gráficas, redes globales, fin del escalado de frecuencia mononúcleo (Dennard scaling), vulnerabilidades de memoria en sistemas críticos, contenedores cloud y compilación a WebAssembly.

Consejo

No confunda la popularidad circunstancial de un lenguaje con superioridad técnica universal: cada lenguaje encarna compromisos deliberados (por ejemplo, Go sacrifica expresividad de tipos algebraicos y control sin GC a cambio de simplicidad de runtime y velocidad de compilación; Rust invierte el compromiso exigiendo demostrar la validez de los préstamos en tiempo de compilación para eliminar el GC).

Definición

Principio de Abstracción de Coste Cero (Zero-Overhead Principle) y Seguridad Temporal/Espacial de Memoria: una abstracción cumple coste cero cuando compila a instrucciones máquina tan eficientes como el código de bajo nivel escrito manualmente. La seguridad espacial garantiza que ningún puntero acceda fuera de los límites del objeto asignado; la seguridad temporal garantiza que ninguna referencia apunte a memoria ya liberada (ausencia de dangling pointers o use-after-free).

Autoficha

1. ¿Qué diferencia arquitectónica separa la herencia de clases de C++/Java de la delegación por prototipos de JavaScript? 2. ¿Cómo resuelve Rust la seguridad temporal y espacial de memoria sin utilizar recolector de basura en tiempo de ejecución? 3. ¿Por qué el fin del escalado de Dennard impulsó el auge de modelos como goroutines, async/await, actores e inmutabilidad funcional?

3.1 Smalltalk, C++, Java, Python, JavaScript, C# y el movimiento del software libre

A medida que los sistemas superaron el millón de líneas de código en los años setenta y ochenta, el cuello de botella dejó de ser expresar bucles individuales y pasó a ser organizar subsistemas desacoplados. Simula 67 (Ole-Johan Dahl y Kristen Nygaard) introdujo clases, objetos, subtipos y corrutinas para simulación de eventos discretos. Alan Kay en Xerox PARC generalizó la idea en Smalltalk-80, donde toda entidad es un objeto que se comunica exclusivamente mediante paso de mensajes dinámico. En los Laboratorios Bell, Bjarne Stroustrup creó C with Classes (1979) y posteriormente C++ (1985), combinando la estructuración de Simula con el control de bajo nivel de C bajo el principio de abstracción de coste cero (zero-overhead principle): lo que no se usa no se paga, y lo que se usa no debería poder escribirse más eficientemente a mano. Los años noventa trajeron la expansión de Internet y la necesidad de portabilidad binaria y productividad. Guido van Rossum publicó Python (1991) priorizando legibilidad, modelo uniforme de objetos y extensibilidad en C. James Gosling en Sun Microsystems presentó Java (1995) con ejecución sobre máquina virtual (JVM), verificación de bytecode y recolección de basura, seguido en 2000 por Anders Hejlsberg en Microsoft con C# y el Common Language Runtime (.NET). También en 1995, Brendan Eich diseñó JavaScript en diez días para Netscape Navigator, integrando funciones de primera clase (inspiradas en Scheme) y herencia prototípica (inspirada en Self), que evolucionaría hasta el estándar ECMAScript moderno y TypeScript (2012). Paralelamente, el proyecto GNU (Richard Stallman, 1983) y el núcleo Linux (Linus Torvalds, 1991), junto con el sistema de control de versiones distribuido Git

(creado por Torvalds en 2005 como grafo acíclico dirigido direccionado por contenido SHA-1/SHA-256), transformaron la colaboración mundial en ingeniería del software.

3.2 Git, paralelismo multinúcleo, Go, Rust, Kotlin, Swift, Cloud, WebAssembly e IA

Alrededor de 2005 se detuvo el escalado térmico de frecuencia de reloj (fin del escalado de Dennard), obligando a la industria a multiplicar núcleos de ejecución y distribuir servicios en centros de datos. Al mismo tiempo, los informes de seguridad de Microsoft y Chromium revelaron que aproximadamente el 70 % de las vulnerabilidades críticas de seguridad en bases de código C y C++ provenían de defectos de seguridad de memoria (use-after-free, double-free, out-of-bounds y carreras de datos). De esta doble presión nacieron los cuatro grandes lenguajes modernos de la década de 2010: Go (Robert Griesemer, Rob Pike y Ken Thompson en Google, 2009), enfocado en compilación ultrarrápida y concurrencia masiva mediante goroutines y canales; Rust (iniciado por Graydon Hoare en Mozilla y estabilizado en 2015), que resolvió el dilema histórico entre control sin recolector de basura y seguridad de memoria en tiempo de compilación mediante un sistema de tipos afín basado en ownership, borrowing y lifetimes; Kotlin (JetBrains, 2011/2016), que erradicó las referencias nulas no controladas en la JVM e introdujo corrutinas estructuradas; y Swift (Chris Lattner en Apple, 2014), basado en semántica de valor, conteo automático de referencias (ARC) y aislamiento de actores en Swift 6. Finalmente, la estandarización de contenedores OCI (Docker/Kubernetes), WebAssembly (Wasm, máquina virtual de pila tipada de velocidad cercana a nativa en navegador y servidor) y los asistentes de IA generativa han elevado aún más la exigencia de verificación determinista: el ingeniero moderno debe dominar invariantes, pruebas automatizadas y arquitectura para auditar cualquier código generado.

Problema resuelto

Enunciado Según la ley de Amdahl, si un servicio de procesamiento tiene una fracción secuencial irreducible $s = 0{,}05$ (5 %) y una fracción paralelizable $p = 0{,}95$ (95 %) que se distribuye sin sobrecarga entre $N = 19$ núcleos de procesador, calcule el factor de aceleración (speedup) $S(N) = \frac{1}{s+p/N}$.

Datos

- $s = 0{,}05$ — Fracción secuencial del programa
- $p = 0{,}95$ — Fracción paralelizable del programa
- $N = 19$ — Número de núcleos de procesamiento

Incógnita Factor de aceleración total S(N) según la ley de Amdahl

Modelo físico

Ley de Amdahl para paralelismo con fracción secuencial fija

$$S = 1/(s + t_p ar)$$

Hipótesis

- Ausencia de sobrecarga adicional de sincronización y comunicación entre hilos

Desarrollo

1. Calculamos el tiempo relativo de la fracción paralela dividida entre los 19 núcleos.

$$t_p ar = p/N$$

$$\frac{0{,}95}{19}$$

2. Sumamos la fracción secuencial y obtenemos el inverso para hallar la aceleración total.

$$S = 1/(s + t_p ar)$$

$$\frac{1}{(0{,}05 + 0{,}05)}$$

Resultado

10

Interpretación Aunque se empleen 19 núcleos, el 5 % secuencial limita la aceleración efectiva a 10x (y su cota asintótica para infinitos núcleos es 1 / 0.05 = 20x), lo que explica por qué minimizar secciones críticas y contención de cerrojos es esencial en concurrencia moderna.

Comprobación de los pasos

- Paso 1: 0,05 — reproducido
- Paso 2: 10 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- La ley de Amdahl asume tamaño de problema fijo; la ley de Gustafson modela cargas que escalan con el número de procesadores.

Errores frecuentes

- Suponer aceleración lineal N sin descontar la serialización en cerrojos (locks) y E/S.

Ejercicios

1 [PRB-0031] (Básico, Conceptual) [VOL01-C03-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0032] (Básico, Numérico) [VOL01-C03-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0033] (Intermedio, Interpretación) [VOL01-C03-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0034] (Intermedio, Aplicación) [VOL01-C03-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0035] (Intermedio, Identificación de errores) [VOL01-C03-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0036] (Avanzado, Desarrollo por pasos) [VOL01-C03-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0037] (Avanzado, Comparación) [VOL01-C03-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria

segura: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0038] (Avanzado, Comprobación de hipótesis) [VOL01-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0039] (Nivel de examen, Optimización) [VOL01-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0040] (Nivel de examen, Demostración) [VOL01-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0041] (Reto, Diseño) [VOL01-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0042] (Reto, Estudio de parámetros) [VOL01-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Parte II

PARTE II. Fundamentos Universales de Programación y Matemáticas Discretas

Capítulo 4

Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

Dominar cómo toda información —enteros con signo, números reales aproximados y texto multilingüe— se codifica físicamente mediante secuencias finitas de bits, por qué aparecen errores como `0.1 + 0.2 != 0.3` o el desbordamiento entero, y cómo procesarlos con exactitud en cualquier lenguaje.

Consejo

Jamás compare números en coma flotante IEEE 754 mediante igualdad exacta `a == b` tras cálculos iterativos: utilice tolerancia mixta $|a-b| \leq \max(\varepsilon_{\text{rel}} \max(|a|,|b|), \varepsilon_{\text{abs}})$ y recuerde que `NaN != NaN` por definición del estándar (lo que explica por qué en Rust `f32`/`f64` implementan `PartialEq` pero no la relación de equivalencia total `Eq`).

Definición

Épsilon de Máquina ($\varepsilon_{\text{mach}}$) y Codificación Prefijo Libre de Auto-Sincronización (UTF-8): el épsilon de máquina en IEEE 754 es la distancia entre 1,0 y el siguiente número en coma flotante representable ($2^{-23} \approx 1{,}19 \times 10^{-7}$ en `binary32` y $2^{-52} \approx 2{,}22 \times 10^{-16}$ en `binary64`). UTF-8 es una codificación prefijo auto-sincronizante porque los patrones de bits iniciales (`0xxxxxxx`, `11xxxxxx`, `10xxxxxx`) permiten localizar el inicio de cualquier punto de código desde cualquier desplazamiento arbitrario en a lo sumo 3 octetos.

Autoficha

1. ¿Por qué el valor absoluto `abs(INT_MIN)` desborda en complemento a dos de ancho fijo? 2. ¿Por qué $0{,}1 + 0{,}2 \neq 0{,}3$ en coma flotante binaria IEEE 754 pero $0{,}5 + 0{,}25 = 0{,}75$ es exacto? 3. ¿Por qué indexar una cadena UTF-8 por el k-ésimo punto de código requiere tiempo $O(k)$ mientras que indexar por byte es $O(1)$?

4.1 Bits, bytes, endianness, enteros sin signo y complemento a dos

En el nivel físico y arquitectónico, la memoria de un computador es una sucesión direccionable de octetos (bytes de 8 bits). Un vector de w bits representa un entero sin signo en el rango $[0, 2^w - 1]$ mediante evaluación polinómica en base 2: $U(\vec{b}) = \sum_{i=0}^{w-1} b_i 2^i$. Para representar enteros con signo sin duplicar los circuitos sumadores ni sufrir el doble cero de las representaciones signo-magnitud o complemento a uno, la práctica totalidad de las arquitecturas modernas utiliza complemento a dos, donde el bit más significativo b_{w-1} tiene peso negativo -2^{w-1}: $T(\vec{b}) = -b_{w-1}2^{w-1} + \sum_{i=0}^{w-2} b_i 2^i$. El rango resultante es asimétrico, $[-2^{w-1}, 2^{w-1}-1]$ (para $w = 8$, $[-128, 127]$), lo que implica que el valor mínimo $T_{\text{mín}} = -2^{w-1}$ no tiene opuesto representable en el mismo ancho de palabra. Cuando una palabra de múltiples bytes (como un `uint32_t` de valor `0x12345678`) se almacena en direcciones consecutivas de memoria, el orden de los octetos viene dado por el endianness: en arquitecturas little-endian (x86-64, ARM64 por defecto, RISC-V), el byte menos significativo `0x78` reside en la dirección más baja; en protocolos de red TCP/IP (network byte order, big-endian), el byte más significativo `0x12` se transmite primero.

4.2 Aritmética IEEE 754 y codificación de longitud variable UTF-8

Para aproximar números reales con rango dinámico amplio, el estándar IEEE 754 (cuyo formato `binary32` se muestra en la figura 2) representa un número normalizado como $(-1)^s \times (1 + M/2^p) \times 2^{E-\text{bias}}$, donde el

sesgo es bias = 127 en precisión simple (`f32`, $p = 23$) y bias = 1023 en precisión doble (`f64`, $p = 52$). Dado que la fracción 0,1 = 1/10 tiene denominador con factor primo 5, su desarrollo en base 2 es periódico infinito ($0{,}0001100110011\ldots_2$); al redondearse a 53 bits significativos en `binary64`, la suma `0.1 + 0.2` difiere de `0.3` en $2^{-54} \approx 5{,}55 \times 10^{-17}$. Por ello, la aritmética monetaria nunca debe usar coma flotante binaria, sino enteros de céntimos o tipos decimales. En cuanto al texto, Unicode asigna a cada carácter un punto de código escalar (code point) en $[0, \text{U+10FFFF}]$. La codificación UTF-8 (RFC 3629, diseñada por Ken Thompson y Rob Pike) serializa cada escalar en 1 a 4 octetos: los caracteres ASCII (`U+0000..U+007F`) ocupan 1 byte `0xxxxxxx`, garantizando retrocompatibilidad estricta; las secuencias multibyte comienzan con un prefijo `110xxxxx`, `1110xxxx` o `11110xxx` seguido de bytes de continuación `10xxxxxx`, de modo que ningún byte ASCII aparece jamás dentro de un carácter multibyte. Verifiquemos ambos principios en Python:

```
import struct

# IEEE 754 binary32 de -3.5 = (-1)^1 * (1 + 0.75) * 2^(128 - 127)
raw_bits = struct.unpack('>I', struct.pack('>f', -3.5))[0]
assert raw_bits == 0xC0600000

# Codificación UTF-8 de 'Ñ' (U+00D1 -> 2 bytes) y 'U+1F980' (U+1F980 -> 4 bytes)
assert 'Ñ'.encode('utf-8') == b'\xc3\x91'
assert len('U+1F980'.encode('utf-8')) == 4
```

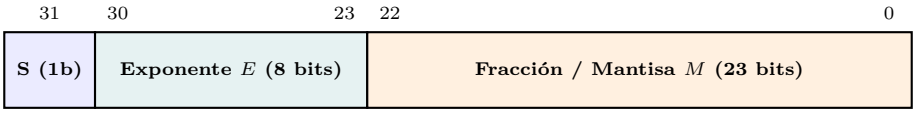


Figura 2. Estructura de bits de un número en coma flotante IEEE 754 de precisión simple (binary32: 1 bit de signo, 8 bits de exponente sesgado por 127 y 23 bits de mantisa fraccionaria).

Problema resuelto

Enunciado Decodifique manualmente el valor real exacto representado por la palabra IEEE 754 `binary32 0xC1580000`, sabiendo que su bit de signo es $s = 1$, su campo de exponente de 8 bits vale $E = 130$ (con sesgo bias = 127) y los tres primeros bits de su mantisa fraccionaria son `101` ($M/2^{23} = 2^{-1} + 2^{-3} = 0{,}625$).

Datos

- $s = 1$ — Bit de signo (negativo)
- $E = 130$ — Exponente almacenado sin signo
- $f = 0{,}625$ — Fracción de mantisa $M/2^23$

Incógnita Valor numérico real exacto representado por la palabra IEEE 754

Modelo físico

Fórmula de evaluación de número normalizado IEEE 754 binary32

$$val = -1 * sig * 2^(130 - 127)$$

Hipótesis

- El exponente E = 130 pertenece al rango normalizado 1 <= E <= 254

Desarrollo

1. Calculamos el significando normalizado sumando el bit implícito 1 a la fracción de mantisa.

$$sig = 1 + 0{,}5 + 0{,}125$$

$$((1 + 0{,}5) + 0{,}125)$$

2. Aplicamos el signo $(-1)^1$ y multiplicamos por la potencia 2^(130 - 127) = 8.

$$val = -1 * sig * 2^(130 - 127)$$

$$\left((-(1) \cdot 1{,}625) \cdot (2)^{(130-127)}\right)$$

Resultado

$$-13$$

Interpretación La palabra `0xC1580000` representa exactamente el número `-13.0`, ya que $-1{,}625 \times 8 = -13$.

Comprobación de los pasos

- Paso 1: 1,625 — reproducido
- Paso 2: -13 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- En números subnormales (E = 0), el bit implícito pasa a ser 0 y el exponente efectivo se fija en 1 - bias = -126.

Errores frecuentes

- Olvidar sumar el bit implícito `1.` del significando en números normalizados.

Ejercicios

1 [PRB-0045] (Básico, Conceptual) [VOL01-C04-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0046] (Básico, Numérico) [VOL01-C04-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0047] (Intermedio, Interpretación) [VOL01-C04-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0048] (Intermedio, Aplicación) [VOL01-C04-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0049] (Intermedio, Identificación de errores) [VOL01-C04-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0050] (Avanzado, Desarrollo por pasos) [VOL01-C04-E06] (Nivel 3– Resolución (Im-

plementación multi-etapa con invariante)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0051] (Avanzado, Comparación) [VOL01-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0052] (Avanzado, Comprobación de hipótesis) [VOL01-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0053] (Nivel de examen, Optimización) [VOL01-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0054] (Nivel de examen, Demostración) [VOL01-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0055] (Reto, Diseño) [VOL01-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0056] (Reto, Estudio de parámetros) [VOL01-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 5

Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

Dotar al lector de las herramientas de matemática discreta que sustentan el razonamiento sobre programas: simplificación de predicados, demostración de corrección de algoritmos recursivos e iterativos mediante inducción, modelado de dependencias mediante órdenes parciales y grafos.

Consejo

Cuando implemente operadores de igualdad o comparación personalizados en cualquier lenguaje, verifique explícitamente los axiomas matemáticos: violar la transitividad en un comparador (`cmp`) provoca resultados corruptos o pánicos en algoritmos de ordenación como Timsort o pdqsort.

Definición

Relación de Equivalencia, Orden Parcial y Lema del Apretón de Manos: una relación binaria $\sim$ sobre A es de equivalencia si es reflexiva ($a \sim a$), simétrica ($a \sim b \Rightarrow b \sim a$) y transitiva ($a \sim b \wedge b \sim c \Rightarrow a \sim c$). Es un orden parcial ($\preceq$) si es reflexiva, antisimétrica ($a \preceq b \wedge b \preceq a \Rightarrow a = b$) y transitiva. En cualquier grafo no dirigido $G = (V, E)$, $\sum_{v \in V} \deg(v) = 2|E|$.

Autoficha

1. ¿Por qué el tipo `f64` en IEEE 754 viola la reflexividad de una relación de equivalencia para el valor `NaN`? 2. ¿Cómo se transforma la negación de una condición compuesta `!(autenticado && (es_admin || tiene_token))` usando De Morgan? 3. ¿Por qué un grafo de dependencias de módulos debe ser un grafo acíclico dirigido (DAG) para admitir orden topológico?

5.1 Álgebra booleana, lógica formal e inducción matemática y estructural

Cada condición `if`, filtro de consulta SQL o guarda de concurrencia es una fórmula de la lógica proposicional y de primer orden. Dos fórmulas P y Q son lógicamente equivalentes ($P \equiv Q$) cuando toman el mismo valor de verdad bajo toda asignación de sus variables. Las leyes de De Morgan, $\neg(P \wedge Q) \equiv (\neg P) \vee (\neg Q)$ y $\neg(P \vee Q) \equiv (\neg P) \wedge (\neg Q)$, junto con la equivalencia de la implicación $(P \Rightarrow Q) \equiv (\neg P \vee Q)$ y su contrarrecíproca $(\neg Q \Rightarrow \neg P)$, permiten refactorizar condiciones anidadas hacia cláusulas de guarda (early returns) legibles y libres de defectos. La inducción matemática y su generalización a tipos de datos recursivos (inducción estructural) son el fundamento de toda prueba de corrección. Para demostrar que una propiedad $P(t)$ se cumple para todo árbol binario t, basta probar el caso base para la hoja vacía y demostrar el paso inductivo: asumiendo la hipótesis de inducción $P(\text{izq})$ y $P(\text{der})$, probar que se cumple $P(\text{Nodo}(v, \text{izq}, \text{der}))$. Esta correspondencia directa entre estructura de datos inductiva y función recursiva guía el diseño con pattern matching en Rust, Haskell y OCaml.

5.2 Conjuntos, relaciones, funciones, combinatoria y grafos en computación

Las estructuras discretas modelan directamente los objetos del software. Un conjunto finito sin duplicados respalda `HashSet`/`BTreeSet`; una relación binaria $R \subseteq A \times A$ que es reflexiva, simétrica y transitiva define una relación de equivalencia (el contrato exacto que exige el trait `Eq` en Rust o el método `equals` en Java),

mientras que una relación reflexiva, antisimétrica y transitiva define un orden parcial (`PartialOrd`), base de la ordenación topológica de dependencias en gestores de paquetes (Cargo, npm, pip) y sistemas de construcción (Make, Bazel). Un grafo $G = (V, E)$ modela redes de computadores, grafos de llamadas de un compilador, referencias de memoria de un recolector de basura o el historial de commits de Git. Por el lema del apretón de manos (Handshaking Lemma), en todo grafo no dirigido la suma de los grados de todos los vértices es exactamente el doble del número de aristas: $\sum_{v \in V} \deg(v) = 2|E|$. Podemos comprobar exhaustivamente las leyes booleanas y el lema del apretón de manos mediante un script verificable:

```
# Verificación exhaustiva de De Morgan sobre {False, True}^2
for p in (False, True):
    for q in (False, True):
        assert (not (p and q)) == ((not p) or (not q))
        assert (not (p or q)) == ((not p) and (not q))

# Lema del apretón de manos en un grafo no dirigido
edges = [(0, 1), (0, 2), (1, 2), (2, 3), (3, 4)]
deg = {v: 0 for v in range(5)}
for u, v in edges:
    deg[u] += 1
    deg[v] += 1
assert sum(deg.values()) == 2 * len(edges) == 10
```

Problema resuelto

Enunciado En una arquitectura de microservicios con $n = 12$ nodos interconectados en topología de malla completa no dirigida K_{12} (donde cada par de nodos distintos mantiene un canal bidireccional), calcule el número total de canales $E = \binom{12}{2} = \frac{n(n-1)}{2}$ y verifique la suma de grados de todos los vértices.

Datos

- $n = 12$ — Número de vértices del grafo completo K_12

Incógnita Número total de aristas (canales bidireccionales) E

Modelo físico

Combinatoria de pares sin orden en un grafo simple completo K_n

$$E = sum_deg/2$$

Hipótesis

- Grafo simple sin bucles propios (self-loops) ni aristas múltiples entre el mismo par de nodos

Desarrollo

1. Multiplicamos el número de nodos n = 12 por los n - 1 = 11 vecinos de cada nodo (suma de grados 2E).

$$sum_deg = n * (n - 1)$$

$$(12 \cdot 11)$$

2. Dividimos entre 2 aplicando el lema del apretón de manos para obtener el número de aristas.

$$E = sum_deg/2$$

$$\frac{132}{2}$$

Resultado

66

Interpretación El crecimiento cuadrático $O(n^2)$ de una malla completa (66 canales para 12 nodos, 4950 para 100 nodos) explica por qué los sistemas distribuidos grandes utilizan topologías jerárquicas, buses de eventos o protocolos gossip.

Comprobación de los pasos

- Paso 1: 132 — reproducido
- Paso 2: 66 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- En comunicación dirigida asimétrica (dígrafo completo), el número de enlaces dirigidos es n(n - 1) = 132.

Errores frecuentes

- Olvidar dividir entre 2 al contar aristas no dirigidas, duplicando cada par {u, v}.

Ejercicios

1 [PRB-0059] (Básico, Conceptual) [VOL01-C05-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0060] (Básico, Numérico) [VOL01-C05-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0061] (Intermedio, Interpretación) [VOL01-C05-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0062] (Intermedio, Aplicación) [VOL01-C05-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0063] (Intermedio, Identificación de errores) [VOL01-C05-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0064] (Avanzado, Desarrollo por pasos) [VOL01-C05-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0065] (Avanzado, Comparación) [VOL01-C05-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice

el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0066] (Avanzado, Comprobación de hipótesis) [VOL01-C05-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0067] (Nivel de examen, Optimización) [VOL01-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0068] (Nivel de examen, Demostración) [VOL01-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0069] (Reto, Diseño) [VOL01-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0070] (Reto, Estudio de parámetros) [VOL01-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 6

Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

Construir un modelo mental preciso de qué ocurre físicamente desde que se escribe un archivo de código fuente hasta que la CPU ejecuta instrucciones máquina y mueve líneas de caché entre la RAM y los registros.

Consejo

Al razonar sobre el rendimiento de cualquier estructura de datos, recuerde que un fallo de caché a RAM ($\sim$ 80 ns) cuesta lo mismo que más de 200 instrucciones aritméticas en registros: un vector contiguo (`Vec<T>`, `std::vector<T>`) suele superar ampliamente a una lista enlazada incluso en inserciones intermedias moderadas gracias a la localidad espacial y al prefetcher de hardware.

Definición

Tiempo Medio de Acceso a Memoria (AMAT) y Marco de Activación (Stack Frame): el tiempo medio de acceso a memoria en una jerarquía con tiempo de acierto en caché T_{hit}, tasa de fallos m (miss rate) y penalización por fallo T_{miss} viene dado por $\text{AMAT} = T_{\text{hit}} + m \cdot T_{\text{miss}}$. El marco de activación es el bloque contiguo de la pila de llamadas asociado a una invocación activa de subrutina.

Autoficha

1. ¿Por qué reservar y liberar memoria en la pila (stack) es mucho más rápido que en el montículo (heap)? 2. ¿Qué ventajas e inconvenientes presenta la compilación AOT (Rust/C++) frente a la compilación JIT con perfilado adaptativo (JVM/V8)? 3. ¿Cómo influye la tasa de fallos de caché m en el tiempo medio de acceso a memoria (AMAT)?

6.1 CPU, registros, pipeline, memoria principal, pila y montículo (stack y heap)

Un procesador moderno ejecuta un flujo continuo de instrucciones gobernado por el contador de programa (`PC` o `RIP`/`PC` en x86-64 y ARM64), un conjunto pequeño de registros ultrarrápidos (generalmente 16 o 32 registros de propósito general de 64 bits con acceso en $\sim$ 0,3 ns), unidades aritmético-lógicas y vectoriales (SIMD), y una jerarquía de memorias caché (L1 de $\sim$ 1 ns, L2 de $\sim$ 4 ns, L3 de $\sim$ 12 ns) que amortiguan la latencia de la memoria principal DRAM ($\sim$ 60–100 ns). El espacio de direcciones virtuales de un proceso se divide convencionalmente en segmentos de código (`.text`), datos estáticos (`.rodata`, `.data`, `.bss`), el montículo dinámico (heap, gestionado por un asignador o recolector de basura) y la pila de ejecución (call stack). La pila de ejecución crece y decrece en orden LIFO (Last-In, First-Out) mediante un único registro puntero de pila (`SP`). Cada llamada a función reserva instantáneamente un marco de activación (stack frame) desplazando `SP`, donde almacena la dirección de retorno, registros preservados y variables locales de tamaño conocido en tiempo de compilación. Al retornar, basta restaurar `SP`, por lo que la reserva y liberación en pila tiene coste $O(1)$ de una sola instrucción y localidad espacial óptima en caché L1. El heap, en cambio, permite reservar bloques cuyo tamaño se decide en tiempo de ejecución o cuya vida útil sobrevive a la función que los creó, a cambio de gestionar metadatos de bloques libres, fragmentación e indirección por punteros.

6.2 De código fuente a ejecución: interpretación, bytecode, compilación AOT y JIT

Para transformar código fuente en acciones del procesador existen tres estrategias principales y sus híbridos. Un intérprete puro analiza el código y evalúa directamente el árbol sintáctico o un bytecode intermedio mediante un bucle de despacho (como CPython o MRI Ruby), ofreciendo arranque inmediato y portabilidad, pero pagando sobrecarga de decodificación y comprobación dinámica de tipos en cada operación. Un compilador Ahead-of-Time (AOT), como `rustc`, `clang` o `gcc`, traduce el programa completo antes de su ejecución a través de análisis léxico, sintáctico, semántico, optimización sobre representación intermedia SSA (Static Single Assignment, p. ej. LLVM IR) y generación de código máquina nativo específico para la arquitectura destino. Las plataformas con compilación Just-in-Time (JIT), como HotSpot JVM (Java/Kotlin), CoreCLR (.NET/C#), V8 (JavaScript) o LuaJIT, combinan ambos mundos: compilan primero a un bytecode portable que comienza ejecutándose en un intérprete o compilador base rápido y, a medida que el perfilador en vivo (profiler) detecta funciones calientes (hot spots), un compilador optimizador en segundo plano genera código máquina nativo especializado con desvirtualización e inlining especulativo.

Problema resuelto

Enunciado Un procesador tiene una caché L1 con tiempo de acierto $T_{\text{hit}} = 1$ ns y una penalización de acceso a memoria principal en caso de fallo de $T_{\text{miss}} = 80$ ns. Calcule el Tiempo Medio de Acceso a Memoria (AMAT $= T_{\text{hit}} + m \cdot T_{\text{miss}}$) en nanosegundos cuando una estructura contigua logra una tasa de fallos de $m = 0{,}025$ (2.5 %).

Datos

- $T_{hit} = 1$ — Latencia de acierto en caché L1 (ns)
- $m = 0{,}025$ — Tasa de fallos de caché (miss rate)
- $T_{miss} = 80$ — Penalización por fallo hacia DRAM (ns)

Incógnita Tiempo medio de acceso a memoria AMAT en nanosegundos

Modelo físico

Ecuación fundamental de rendimiento de jerarquía de caché (AMAT)

$$AMAT = T_hit + T_penalty$$

Hipótesis

- Modelo de un nivel efectivo entre caché L1 y memoria principal

Desarrollo

1. Calculamos la penalización media ponderada por la tasa de fallos del 2.5 %.

$$T_penalty = m * T_miss$$

$$(0{,}025 \cdot 80)$$

2. Sumamos el tiempo base de acierto en L1 para obtener el AMAT en nanosegundos.

$$AMAT = T_hit + T_penalty$$

$$(1 + 2)$$

Resultado

$$3$$

Interpretación Con un 97.5 % de aciertos, el acceso efectivo es de solo 3 ns; si se recorriera una lista enlazada dispersa con m = 0.50 (50 % de fallos), el AMAT subiría a 1 + 40 = 41 ns (casi 14 veces más lento).

Comprobación de los pasos

- Paso 1: 2 — reproducido
- Paso 2: 3 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Los procesadores superescalares fuera de orden pueden solapar parte de la latencia mediante ejecución especulativa y MLP (Memory-Level Parallelism).

Errores frecuentes

- Despreciar el término $m*T_miss$ pensando que un 2.5 % de fallos aporta solo un 2.5 % del tiempo total.

Ejercicios

1 [PRB-0073] (Básico, Conceptual) [VOL01-C06-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0074] (Básico, Numérico) [VOL01-C06-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0075] (Intermedio, Interpretación) [VOL01-C06-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0076] (Intermedio, Aplicación) [VOL01-C06-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

5 [PRB-0077] (Intermedio, Identificación de errores) [VOL01-C06-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

6 [PRB-0078] (Avanzado, Desarrollo por pasos) [VOL01-C06-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

7 [PRB-0079] (Avanzado, Comparación) [VOL01-C06-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0080] (Avanzado, Comprobación de hipótesis) [VOL01-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0081] (Nivel de examen, Optimización) [VOL01-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0082] (Nivel de examen, Demostración) [VOL01-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0083] (Reto, Diseño) [VOL01-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0084] (Reto, Estudio de parámetros) [VOL01-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 7

Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

Comprender cómo los sistemas de tipos previenen estados inválidos antes de ejecutar el software, cómo funcionan las reglas de alcance léxico y tiempo de vida de las variables, y cómo elegir entre valores de retorno explícitos (`Result<T, E>`) y excepciones según el dominio.

Consejo

Reemplace pares de banderas booleanas y campos opcionales sueltos por un tipo suma (`enum` o unión discriminada): un registro con `is_loading: bool`, `data: Option<D>` y `error: Option<E>` multiplica estados imposibles (como `is_loading = true` con `data` y `error` simultáneamente presentes), mientras que un `enum RequestState { Idle, Loading, Ok(D), Failed(E) }` elimina de raíz esos estados en compilación.

Definición

Álgebra de Tipos (Tipos Producto y Tipos Suma): dado un conjunto de tipos con cardinalidades finitas, un tipo producto $T_{\text{prod}} = A \times B$ (registro o tupla) tiene cardinalidad $|A \times B| = |A| \cdot |B|$, mientras que un tipo suma etiquetado $T_{\text{sum}} = A + B$ (unión discriminada o `enum`) tiene cardinalidad $|A + B| = |A| + |B|$. Por ejemplo, $|\text{Option}\langle A\rangle| = |A| + 1$ y $|\text{Result}\langle A, E\rangle| = |A| + |E|$.

Autoficha

1. ¿Cuál es la diferencia exacta entre ámbito léxico (scope) y tiempo de vida en memoria (lifetime)? 2. ¿Por qué un tipo producto multiplica cardinalidades mientras que un tipo suma las adiciona? 3. ¿Qué ventaja ofrece devolver `Result<T, E>` en la firma frente a lanzar excepciones no comprobadas?

7.1 Taxonomía rigurosa de sistemas de tipos: estático, dinámico, nominal, estructural y algebraico

Un sistema de tipos es un método sintáctico tractable para demostrar la ausencia de ciertos comportamientos erróneos clasificando las expresiones según la naturaleza de los valores que calculan (Benjamin Pierce, Types and Programming Languages). Conviene distinguir con precisión tres ejes ortogonales que a menudo se confunden: 1. Momento de verificación (estático vs dinámico): en lenguajes de verificación estática (Rust, C, C++, Java, C#, Go, Kotlin, Swift, TypeScript, Haskell), los tipos de las expresiones se resuelven y comprueban en tiempo de compilación; en lenguajes dinámicos (Python, JavaScript, Ruby, Lua, Clojure), las etiquetas de tipo están asociadas a los valores en tiempo de ejecución. 2. Solidez y coerciones (fuerte/seguro vs débil/inseguro): un sistema de tipos es seguro (type-safe y memory-safe) si impide tratar un patrón arbitrario de bits como si perteneciera a otro tipo incompatible y prohíbe conversiones implícitas con pérdida silenciosa. 3. Equivalencia y subtipado (nominal vs estructural): en el tipado nominal (Rust, Java, C++, C#, Swift), dos tipos son compatibles solo si comparten declaración explícita de nombre o implementación de trait; en el tipado estructural (TypeScript, interfaces de Go), la compatibilidad depende exclusivamente de la forma (shape) de sus campos o métodos. Finalmente, los Tipos de Datos Algebraicos (ADT) combinan tipos producto (`struct`/tuplas, cuya cardinalidad es $|A \times B| = |A| \cdot |B|$) y tipos suma discriminados (`enum` en Rust/Swift, variantes en OCaml/Haskell, uniones discriminadas en TypeScript, cuya cardinalidad es $|A + B| = |A| + |B|$), permitiendo hacer irrepresentables los estados inválidos.

7.2 Ámbito léxico, duración, clausuras, acoplamiento modular y gestión de errores

Es imprescindible distinguir entre el ámbito (scope) de un identificador y el tiempo de vida (lifetime / storage duration) del valor al que se vincula. El ámbito léxico es una propiedad estática del texto del programa: determina en qué región de código es visible un nombre. El tiempo de vida es una propiedad dinámica de ejecución: el intervalo entre la creación y la destrucción del objeto en memoria. Una clausura (closure) aparece cuando una función de primera clase captura variables libres de su entorno léxico circundante, empaquetando el puntero de código junto con un registro de entorno. En cuanto a la gestión de fallos, la ingeniería del software distingue entre defectos de programación (violaciones de invariantes internos, ante los cuales procede abortar o activar un panic/assertion) y errores operativos recuperables (archivo inexistente, timeout de red, entrada mal formada). Los lenguajes modernos modelan los errores recuperables bien mediante excepciones estructuradas (`try/catch` en Python, Java, C#, C++, JS) o bien como valores explícitos en la firma de retorno (`Result<T, E>` en Rust, tupla `(T, error)` en Go), obligando al llamador a tratar cada rama de fallo explícitamente:

```rust
#[derive(Debug, PartialEq)]
enum ParsePortError {
    NotANumber,
    ZeroPortForbidden,
}

fn parse_port(raw: &str) -> Result<u16, ParsePortError> {
    let port: u16 = raw.trim().parse().map_err(|_| ParsePortError::NotANumber)?;
    if port == 0 {
        return Err(ParsePortError::ZeroPortForbidden);
    }
    Ok(port)
}

fn main() {
    assert_eq!(parse_port("8080"), Ok(8080));
    assert_eq!(parse_port("0"), Err(ParsePortError::ZeroPortForbidden));
}
```

Problema resuelto

Enunciado Un estado de conexión se modela erróneamente como un tipo producto con tres booleanos (`conectado`, `autenticado`, `error_activo`), un código de modo de 4 valores posibles y un canal opcional de $8 + 1 = 9$ estados posibles. Compare el número de estados del tipo producto $N_{\text{prod}} = 2^3 \times 4 \times 9$ frente a un tipo suma discriminado `ConnectionState` con 4 variantes exactas: `Disconnected` (1), `Connecting` (4 modos), `Authenticated` ($4 \times 8 = 32$ combinaciones válidas de modo y canal activo) y `Failed` (1), y calcule cuántos estados ilegales elimina el diseño con tipos algebraicos.
Datos

- $N_{bool} = 8$ — Combinaciones de los 3 booleanos (2^3)
- $N_{mode} = 4$ — Modos de operación posibles
- $N_{chan} = 9$ — Estados de Option<Channel> $(8 + 1)$

Incógnita Número de estados inválidos eliminados al pasar de tipo producto a tipo suma
Modelo físico
Cardinalidad de tipos algebraicos (producto cartesiano frente a unión disjunta)

$$N_invalid = N_prod - (1 + 4 + 32 + 1)$$

Hipótesis

- Solo las 38 configuraciones del tipo suma tienen significado válido en el protocolo

Desarrollo

1. Calculamos la cardinalidad total del registro tipo producto.

$$N_prod = 8 * 4 * 9$$

$$((8 \cdot 4) \cdot 9)$$

2. Calculamos la cardinalidad del tipo suma ($1 + 4 + 32 + 1 = 38$) y la restamos de N_prod.

$$N_invalid = N_prod - (1 + 4 + 32 + 1)$$

$$(288 - (((1 + 4) + 32) + 1))$$

Resultado

$$250$$

Interpretación El registro plano permite representar 288 estados posibles de los cuales 250 son combinaciones contradictorias; el tipo suma reduce la cardinalidad exactamente a los 38 estados válidos.

Comprobación de los pasos

- Paso 1: 288 — reproducido
- Paso 2: 250 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Limitaciones

- Invariantes numéricos continuos (p. ej. $x^2 + y^2 = 1$) requieren constructores inteligentes (newtypes) además de tipos suma.

Errores frecuentes

- Usar el valor nulo o punteros sin inicializar como centinela implícito en lugar de `Option<T>`.

Ejercicios

1 [PRB-0087] (Básico, Conceptual) [VOL01-C07-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

2 [PRB-0088] (Básico, Numérico) [VOL01-C07-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

3 [PRB-0089] (Intermedio, Interpretación) [VOL01-C07-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

4 [PRB-0090] (Intermedio, Aplicación) [VOL01-C07-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(25 + 8) * 2` y justifique los criterios de aceptación,

pistas graduadas y complejidad temporal/espacial.
5 [PRB-0091] (Intermedio, Identificación de errores) [VOL01-C07-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0092] (Avanzado, Desarrollo por pasos) [VOL01-C07-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0093] (Avanzado, Comparación) [VOL01-C07-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
8 [PRB-0094] (Avanzado, Comprobación de hipótesis) [VOL01-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
9 [PRB-0095] (Nivel de examen, Optimización) [VOL01-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
10 [PRB-0096] (Nivel de examen, Demostración) [VOL01-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
11 [PRB-0097] (Reto, Diseño) [VOL01-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
12 [PRB-0098] (Reto, Estudio de parámetros) [VOL01-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Capítulo 8

Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

Establecer un marco comparativo riguroso de los doce paradigmas de programación fundamentales, comprendiendo que los lenguajes profesionales actuales son multiparadigma y que la excelencia en ingeniería consiste en aplicar el paradigma adecuado a cada capa del sistema.

Consejo

Aplique el patrón arquitectónico Functional Core, Imperative Shell: escriba la lógica de dominio y validación como funciones puras sobre valores inmutables (fáciles de probar exhaustivamente sin mocks), y confine los efectos laterales de entrada/salida, red y base de datos a una capa exterior delgada.

Definición

Transparencia Referencial y Dualidad Concurrencia vs Paralelismo: una expresión es referencialmente transparente si puede sustituirse en cualquier punto del programa por su valor evaluado sin alterar el comportamiento observable del sistema. La concurrencia es una propiedad de diseño (composición de múltiples tareas lógicas independientes que pueden progresar en intervalos solapados), mientras que el paralelismo es una propiedad física de ejecución (varias unidades de hardware ejecutando instrucciones en el mismo instante físico).

Autoficha

1. ¿En qué se diferencia la transparencia referencial funcional del encapsulamiento orientado a objetos? 2. ¿Cuándo es preferible la programación asíncrona basada en bucle de eventos (`async`/`await`) frente a un pool de hilos del sistema operativo? 3. ¿Qué diferencia de ejecución y tamaño binario existe entre los genéricos por monomorfización (Rust/C++) y por borrado de tipos (Java)?

8.1 Imperativo, procedural, orientado a objetos, declarativo, funcional y lógico

Un paradigma de programación es una disciplina conceptual que estructura cómo modelamos el cómputo, el estado y el flujo de control: 1. Programación imperativa: expresa el cómputo como una secuencia explícita de sentencias que mutan un estado almacenado en memoria. Es óptima cuando el algoritmo modela una máquina de estados física o manipula buffers en el lugar (in-place), pero dificulta el razonamiento local cuando el estado mutable se comparte sin control. 2. Programación procedural: organiza el código imperativo en procedimientos y módulos con interfaces bien definidas (C, Pascal, Go). 3. Programación orientada a objetos (OOP): encapsula estado e invariantes detrás de métodos y permite polimorfismo dinámico (despacho dinámico por vtable o paso de mensajes). Su límite aparece cuando se abusa de jerarquías profundas de herencia de implementación en lugar de composición. 4. Programación declarativa: especifica qué propiedades debe satisfacer el resultado sin prescribir el bucle de control paso a paso (SQL en consultas relacionales, HTML/CSS en maquetación, Terraform/Nix en infraestructura). 5. Programación funcional: modela el cómputo como evaluación de funciones matemáticamente puras (transparencia referencial: sin efectos laterales observables y con datos inmutables o persistentes), facilitando la composición, las pruebas unitarias y el paralelismo sin carreras de datos. 6. Programación lógica: expresa hechos y reglas mediante cláusulas de Horn (Prolog, Datalog), delegando la búsqueda de soluciones en un motor de unificación y backtracking.

8.2 Genérico, eventos, reactivo, concurrente, asíncrono y programación de sistemas

Los seis paradigmas restantes abordan la parametrización de tipos, el tiempo, la latencia de E/S y el control de recursos físicos: 7. Programación genérica: parametriza algoritmos y estructuras de datos sobre tipos abstractos restringidos por contratos (`traits` en Rust, `concepts` en C++20, `interfaces` en Go/Java/C#), ejecutándose por monomorfización estática (coste cero en tiempo de ejecución) o por diccionario/borrado. 8. Programación orientada a eventos: estructura el flujo alrededor de un bucle despachador (event loop) que invoca manejadores (handlers/callbacks) en respuesta a sucesos discretos de E/S o interfaz. 9. Programación reactiva: modela flujos de datos asíncronos a lo largo del tiempo (`Flow` en Kotlin, `Stream` en Rust, Rx) con propagación automática de cambios y control de contrapresión (backpressure). 10. Programación concurrente: estructura múltiples flujos de control lógicos potencialmente simultáneos mediante memoria compartida sincronizada (mutexes, atómicos), paso de mensajes por canales (CSP en Go/Rust) o aislamiento de actores (Erlang/Elixir, Swift 6). 11. Programación asíncrona: desacopla la espera de operaciones de entrada/salida (red, disco, temporizadores) del bloqueo de hilos del sistema operativo mediante futuros/promesas y máquinas de estados cooperativas (`async`/`await` sobre `epoll`/`kqueue`/`io_uring`). 12. Programación de sistemas: diseña software que gestiona directamente la memoria física/virtual, la ABI, las llamadas al sistema y las restricciones de latencia determinista (Rust, C, C++, Zig).

Problema resuelto

Enunciado Un servidor debe mantener $C = 10000$ conexiones concurrentes mayoritariamente ociosas esperando E/S de red. Si en un modelo de un hilo del sistema operativo por conexión cada hilo reserva una pila de $S_{\text{thread}} = 1024$ KiB (1 MiB), mientras que en un modelo asíncrono (`async`/`await`) cada tarea es una máquina de estados compilada de $S_{\text{task}} = 2$ KiB, calcule la razón de reducción de memoria $\frac{C \cdot S_{\text{thread}}}{C \cdot S_{\text{task}}}$.

Datos

- $C = 10000$ — Conexiones concurrentes activas
- $S_{thread} = 1024$ — Pila por hilo OS en KiB
- $S_{task} = 2$ — Estado por tarea asíncrona en KiB

Incógnita Factor de reducción de huella de memoria de pila/estado entre ambos modelos

Modelo físico

Comparación de coste espacial entre hilos nativos OS y máquinas de estado asíncronas

$$ratio = M_threads/(C * S_task)$$

Hipótesis

- Cada hilo nativo compromete su pila de 1024 KiB y cada Future compilado ocupa 2 KiB

Desarrollo

1. Calculamos la memoria total en KiB para los 10000 hilos del sistema operativo.

$$M_threads = C * S_thread$$

$$(10000 \cdot 1024)$$

2. Dividimos entre la memoria total de las 10000 tareas asíncronas (20000 KiB).

$$ratio = M_threads/(C * S_task)$$

$$\frac{10240000}{20000}$$

Resultado

512

Interpretación El modelo asíncrono reduce en un factor de 512x la memoria de estado de concurrencia (de ~10 GiB a ~20 MiB) y elimina los cambios de contexto de kernel entre miles de hilos.
Comprobación de los pasos

- Paso 1: 10240000 — reproducido
- Paso 2: 512 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Limitaciones

- Para cargas intensivas en CPU (CPU-bound) sin esperas de E/S, un pool de tantos hilos como núcleos físicos evita la sobrecarga del planificador asíncrono.

Errores frecuentes

- Invocar llamadas bloqueantes sincrónicas dentro de una tarea asíncrona, bloqueando el hilo ejecutor del runtime.

Ejercicios

1 [PRB-0101] (Básico, Conceptual) [VOL01-C08-E01] (Nivel 1– Fundamentos (Comprensión y representación)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
2 [PRB-0102] (Básico, Numérico) [VOL01-C08-E02] (Nivel 1– Fundamentos (Cálculo directo e invariantes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
3 [PRB-0103] (Intermedio, Interpretación) [VOL01-C08-E03] (Nivel 2– Aplicación (Lectura de código y traza de estado)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
4 [PRB-0104] (Intermedio, Aplicación) [VOL01-C08-E04] (Nivel 2– Aplicación (Predicción determinista de salida)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(28 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
5 [PRB-0105] (Intermedio, Identificación de errores) [VOL01-C08-E05] (Nivel 3– Resolución (Detección y corrección de defecto sutil)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
6 [PRB-0106] (Avanzado, Desarrollo por pasos) [VOL01-C08-E06] (Nivel 3– Resolución (Implementación multi-etapa con invariante)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
7 [PRB-0107] (Avanzado, Comparación) [VOL01-C08-E07] (Nivel 4– Ingeniería (Refactorización modular y contratos)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 7, verifique las precondiciones forma-

les, evalúe la métrica característica mediante la expresión reproducible `28 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

8 [PRB-0108] (Avanzado, Comprobación de hipótesis) [VOL01-C08-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

9 [PRB-0109] (Nivel de examen, Optimización) [VOL01-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

10 [PRB-0110] (Nivel de examen, Demostración) [VOL01-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

11 [PRB-0111] (Reto, Diseño) [VOL01-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

12 [PRB-0112] (Reto, Estudio de parámetros) [VOL01-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Apéndice A

Notación

Símbolo	Significado	Unidades
$AMAT$	Tiempo medio de acceso a memoria (Average Memory Access Time)	ns
BNF	Notación formal de Backus-Naur para gramáticas libres de contexto	adimensional
CSP	Communicating Sequential Processes (Hoare, 1978)	adimensional
K_n	Grafo completo no dirigido de n vértices	adimensional
$S(N)$	Factor de aceleración (speedup) con N procesadores	adimensional
Γ	Alfabeto finito de símbolos de cinta	adimensional
δ	Función de transición de la máquina de Turing	adimensional
$\varepsilon_{\text{mach}}$	Épsilon de máquina en coma flotante	adimensional
$\lvert A + B \rvert$	Cardinalidad de un tipo suma discriminado	adimensional
$\lvert A \times B \rvert$	Cardinalidad de un tipo producto	adimensional

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

Problema resuelto [PRB-0001]
Considere una Máquina de Turing determinista con $|Q| = 6$ estados activos, un alfabeto de cinta de $|\Gamma| = 4$ símbolos y $d = 2$ direcciones de movimiento del cabezal. Determine el número total de entradas de la función de transición $\delta : Q \times \Gamma \to (Q \cup \{\text{HALT}\}) \times \Gamma \times \{L, R\}$ y el número de bits necesarios por entrada si cada transición codifica uno de los $6 + 1 = 7$ estados destino (3 bits), el símbolo escrito (2 bits) y la dirección (1 bit), así como el tamaño total en bits de la tabla de transición.
Respuesta

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[VOL01-C01-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 24
Ejercicio 2 [PRB-0004]
[VOL01-C01-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 28
Ejercicio 3 [PRB-0005]
[VOL01-C01-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 4 [PRB-0006]
[VOL01-C01-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(12 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 5 [PRB-0007]
[VOL01-C01-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 6 [PRB-0008]
[VOL01-C01-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 70

Ejercicio 7 [PRB-0009]
[VOL01-C01-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 72
Ejercicio 8 [PRB-0010]
[VOL01-C01-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 87
Ejercicio 9 [PRB-0011]
[VOL01-C01-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 10 [PRB-0012]
[VOL01-C01-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 11 [PRB-0013]
[VOL01-C01-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 145
Ejercicio 12 [PRB-0014]
[VOL01-C01-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre computabilidad, máquinas de Turing y cálculo lambda: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `12 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 144

Cálculos reproducidos

$(24 \cdot 6) = 144$

C.2 Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

Problema resuelto [PRB-0015]
En un computador histórico con memoria de $M = 32768$ palabras de 36 bits, comparar el espacio ocupado por una tabla de $N = 1024$ identificadores de 8 caracteres ASCII (8 bits por carácter) cuando se almacenan como cadenas con centinela nulo de 1 byte frente a cadenas con prefijo de longitud de 4 bytes sin centinela. Calcule el ahorro neto en bytes de la representación con centinela de 1 byte para los $N = 1024$ identificadores.
Respuesta

3072

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0017]
[VOL01-C02-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 30
Ejercicio 2 [PRB-0018]
[VOL01-C02-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 31
Ejercicio 3 [PRB-0019]
[VOL01-C02-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 49
Ejercicio 4 [PRB-0020]
[VOL01-C02-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(15 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 46
Ejercicio 5 [PRB-0021]
[VOL01-C02-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 58
Ejercicio 6 [PRB-0022]
[VOL01-C02-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 85
Ejercicio 7 [PRB-0023]
[VOL01-C02-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 90
Ejercicio 8 [PRB-0024]
[VOL01-C02-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 108
Ejercicio 9 [PRB-0025]
[VOL01-C02-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 132
Ejercicio 10 [PRB-0026]
[VOL01-C02-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 10,

verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 135
Ejercicio 11 [PRB-0027]
[VOL01-C02-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 175
Ejercicio 12 [PRB-0028]
[VOL01-C02-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de ensambladores, compiladores, ALGOL, Lisp y C/Unix: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `15 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 180

Cálculos reproducidos

$(1024 \cdot 3) = 3072$

C.3 De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

Problema resuelto [PRB-0029]
Según la ley de Amdahl, si un servicio de procesamiento tiene una fracción secuencial irreducible $s = 0{,}05$ (5 %) y una fracción paralelizable $p = 0{,}95$ (95 %) que se distribuye sin sobrecarga entre $N = 19$ núcleos de procesador, calcule el factor de aceleración (speedup) $S(N) = \frac{1}{s + p/N}$.
Respuesta

10

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0031]
[VOL01-C03-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 40
Ejercicio 2 [PRB-0032]
[VOL01-C03-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36
Ejercicio 3 [PRB-0033]
[VOL01-C03-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 64
Ejercicio 4 [PRB-0034]
[VOL01-C03-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(20 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 56
Ejercicio 5 [PRB-0035]

[VOL01-C03-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 78
Ejercicio 6 [PRB-0036]
[VOL01-C03-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 110
Ejercicio 7 [PRB-0037]
[VOL01-C03-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120
Ejercicio 8 [PRB-0038]
[VOL01-C03-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 143
Ejercicio 9 [PRB-0039]
[VOL01-C03-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 172
Ejercicio 10 [PRB-0040]
[VOL01-C03-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 180
Ejercicio 11 [PRB-0041]
[VOL01-C03-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 225
Ejercicio 12 [PRB-0042]
[VOL01-C03-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre evolución de paradigmas modernos, ley de Amdahl y memoria segura: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `20 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 240

Cálculos reproducidos

$\frac{1}{(0{,}05+0{,}05)} = 10$

C.4 Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

Problema resuelto [PRB-0043]

Decodifique manualmente el valor real exacto representado por la palabra IEEE 754 `binary32` `0xC1580000`, sabiendo que su bit de signo es $s = 1$, su campo de exponente de 8 bits vale $E = 130$ (con sesgo bias = 127) y los tres primeros bits de su mantisa fraccionaria son `101` ($M/2^{23} = 2^{-1}+2^{-3} = 0{,}625$).
Respuesta

$$-13$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
[VOL01-C04-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 32
Ejercicio 2 [PRB-0046]
[VOL01-C04-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 32
Ejercicio 3 [PRB-0047]
[VOL01-C04-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 52
Ejercicio 4 [PRB-0048]
[VOL01-C04-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(16 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 48
Ejercicio 5 [PRB-0049]
[VOL01-C04-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 62
Ejercicio 6 [PRB-0050]
[VOL01-C04-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 90
Ejercicio 7 [PRB-0051]
[VOL01-C04-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 96
Ejercicio 8 [PRB-0052]
[VOL01-C04-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 115
Ejercicio 9 [PRB-0053]

[VOL01-C04-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 140
Ejercicio 10 [PRB-0054]
[VOL01-C04-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 144
Ejercicio 11 [PRB-0055]
[VOL01-C04-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 185
Ejercicio 12 [PRB-0056]
[VOL01-C04-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre complemento a dos, IEEE 754 y codificación UTF-8: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `16 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 192

Cálculos reproducidos

$$\left(\left(-\left(1\right)\cdot 1{,}625\right)\cdot\left(2\right)^{\left(130-127\right)}\right)=-\left(13\right)$$

C.5 Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

Problema resuelto [PRB-0057]
En una arquitectura de microservicios con $n = 12$ nodos interconectados en topología de malla completa no dirigida K_{12} (donde cada par de nodos distintos mantiene un canal bidireccional), calcule el número total de canales $E = \binom{12}{2} = \frac{n(n-1)}{2}$ y verifique la suma de grados de todos los vértices.
Respuesta

66

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0059]
[VOL01-C05-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 36
Ejercicio 2 [PRB-0060]
[VOL01-C05-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 34
Ejercicio 3 [PRB-0061]

[VOL01-C05-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 58

Ejercicio 4 [PRB-0062]

[VOL01-C05-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(18 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 52

Ejercicio 5 [PRB-0063]

[VOL01-C05-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 70

Ejercicio 6 [PRB-0064]

[VOL01-C05-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 100

Ejercicio 7 [PRB-0065]

[VOL01-C05-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 108

Ejercicio 8 [PRB-0066]

[VOL01-C05-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 129

Ejercicio 9 [PRB-0067]

[VOL01-C05-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 156

Ejercicio 10 [PRB-0068]

[VOL01-C05-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 162

Ejercicio 11 [PRB-0069]

[VOL01-C05-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 205

Ejercicio 12 [PRB-0070]

[VOL01-C05-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre álgebra booleana, inducción estructural, relaciones y grafos: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `18 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 216

Cálculos reproducidos

$\frac{132}{2} = 66$

C.6 Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

Problema resuelto [PRB-0071]
Un procesador tiene una caché L1 con tiempo de acierto $T_{\text{hit}} = 1$ ns y una penalización de acceso a memoria principal en caso de fallo de $T_{\text{miss}} = 80$ ns. Calcule el Tiempo Medio de Acceso a Memoria ($\text{AMAT} = T_{\text{hit}} + m \cdot T_{\text{miss}}$) en nanosegundos cuando una estructura contigua logra una tasa de fallos de $m = 0{,}025$ (2.5 %).
Respuesta

3

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0073]
[VOL01-C06-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 44
Ejercicio 2 [PRB-0074]
[VOL01-C06-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 38
Ejercicio 3 [PRB-0075]
[VOL01-C06-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 70
Ejercicio 4 [PRB-0076]
[VOL01-C06-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(22 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 60
Ejercicio 5 [PRB-0077]
[VOL01-C06-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 86
Ejercicio 6 [PRB-0078]
[VOL01-C06-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 120
Ejercicio 7 [PRB-0079]

[VOL01-C06-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 132
Ejercicio 8 [PRB-0080]
[VOL01-C06-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 157
Ejercicio 9 [PRB-0081]
[VOL01-C06-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 188
Ejercicio 10 [PRB-0082]
[VOL01-C06-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 198
Ejercicio 11 [PRB-0083]
[VOL01-C06-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 245
Ejercicio 12 [PRB-0084]
[VOL01-C06-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre arquitectura de CPU, stack vs heap, AMAT y compilación AOT/JIT: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `22 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 264

Cálculos reproducidos

$(1 + 2) = 3$

C.7 Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

Problema resuelto [PRB-0085]
Un estado de conexión se modela erróneamente como un tipo producto con tres booleanos (`conectado`, `autenticado`, `error_activo`), un código de modo de 4 valores posibles y un canal opcional de $8 + 1 = 9$ estados posibles. Compare el número de estados del tipo producto $N_{\text{prod}} = 2^3 \times 4 \times 9$ frente a un tipo suma discriminado `ConnectionState` con 4 variantes exactas: `Disconnected` (1), `Connecting` (4 modos), `Authenticated` ($4 \times 8 = 32$ combinaciones válidas de modo y canal activo) y `Failed` (1), y calcule cuántos estados ilegales elimina el diseño con tipos algebraicos.
Respuesta

250

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0087]

[VOL01-C07-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 50

Ejercicio 2 [PRB-0088]

[VOL01-C07-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 41

Ejercicio 3 [PRB-0089]

[VOL01-C07-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 79

Ejercicio 4 [PRB-0090]

[VOL01-C07-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(25 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 66

Ejercicio 5 [PRB-0091]

[VOL01-C07-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 98

Ejercicio 6 [PRB-0092]

[VOL01-C07-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 135

Ejercicio 7 [PRB-0093]

[VOL01-C07-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 150

Ejercicio 8 [PRB-0094]

[VOL01-C07-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 178

Ejercicio 9 [PRB-0095]

[VOL01-C07-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 212

Ejercicio 10 [PRB-0096]

[VOL01-C07-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.

Respuesta 225
Ejercicio 11 [PRB-0097]
[VOL01-C07-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 275
Ejercicio 12 [PRB-0098]
[VOL01-C07-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre sistemas de tipos, ADT, ámbito léxico, clausuras y gestión de errores: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `25 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 300

Cálculos reproducidos

$(288 - (((1 + 4) + 32) + 1)) = 250$

C.8 Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

Problema resuelto [PRB-0099]
Un servidor debe mantener $C = 10000$ conexiones concurrentes mayoritariamente ociosas esperando E/S de red. Si en un modelo de un hilo del sistema operativo por conexión cada hilo reserva una pila de $S_{\text{thread}} = 1024$ KiB (1 MiB), mientras que en un modelo asíncrono (`async`/`await`) cada tarea es una máquina de estados compilada de $S_{\text{task}} = 2$ KiB, calcule la razón de reducción de memoria $\frac{C \cdot S_{\text{thread}}}{C \cdot S_{\text{task}}}$.
Respuesta

512

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0101]
[VOL01-C08-E01] (Nivel 1− Fundamentos (Comprensión y representación)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 1, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 56
Ejercicio 2 [PRB-0102]
[VOL01-C08-E02] (Nivel 1− Fundamentos (Cálculo directo e invariantes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 2, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 + 16` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 44
Ejercicio 3 [PRB-0103]
[VOL01-C08-E03] (Nivel 2− Aplicación (Lectura de código y traza de estado)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 3, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 3 + 4` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 88
Ejercicio 4 [PRB-0104]
[VOL01-C08-E04] (Nivel 2− Aplicación (Predicción determinista de salida)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 4, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `(28 + 8) * 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 72
Ejercicio 5 [PRB-0105]

[VOL01-C08-E05] (Nivel 3− Resolución (Detección y corrección de defecto sutil)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 5, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 4 - 2` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 110
Ejercicio 6 [PRB-0106]
[VOL01-C08-E06] (Nivel 3− Resolución (Implementación multi-etapa con invariante)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 6, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 5 + 10` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 150
Ejercicio 7 [PRB-0107]
[VOL01-C08-E07] (Nivel 4− Ingeniería (Refactorización modular y contratos)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 7, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 6` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 168
Ejercicio 8 [PRB-0108]
[VOL01-C08-E08] (Nivel 4− Ingeniería (Validación de precondiciones y bordes)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 8, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 7 + 3` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 199
Ejercicio 9 [PRB-0109]
[VOL01-C08-E09] (Nivel 5− Avanzado (Análisis de rendimiento, memoria y complejidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 9, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 8 + 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 236
Ejercicio 10 [PRB-0110]
[VOL01-C08-E10] (Nivel 5− Avanzado (Demostración de corrección e invariante de bucle)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 10, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 9` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 252
Ejercicio 11 [PRB-0111]
[VOL01-C08-E11] (Nivel 6− Experto (Diseño arquitectónico bajo restricciones reales)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 11, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 10 + 25` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 305
Ejercicio 12 [PRB-0112]
[VOL01-C08-E12] (Nivel 6− Experto (Compromisos técnicos, escalabilidad y mantenibilidad)) Sobre los doce paradigmas de programación, transparencia referencial y concurrencia vs asincronía: analice el escenario técnico número 12, verifique las precondiciones formales, evalúe la métrica característica mediante la expresión reproducible `28 * 12` y justifique los criterios de aceptación, pistas graduadas y complejidad temporal/espacial.
Respuesta 336

Cálculos reproducidos

$\frac{10240000}{20000} = 512$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 25 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 3 respaldada por una fuente, 1 valor o condición.
¶21: dos testimonios. Ecuaciones: 1 reproducida numéricamente, 1 aproximación, 4 valor o condición.
¶22: dos testimonios. Ecuaciones: 2 aproximación.
¶23: dos testimonios. Ecuaciones: 1 valor o condición.
¶24: dos testimonios. Ecuaciones: 1 reproducida numéricamente, 1 valor o condición.
¶25: dos testimonios.
¶26: dos testimonios. Ecuaciones: 4 definición.
¶27: dos testimonios. Ecuaciones: 2 respaldada por una fuente.
¶28: dos testimonios. Ecuaciones: 3 respaldada por una fuente, 6 valor o condición.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios. Ecuaciones: 1 respaldada por una fuente.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios. Ecuaciones: 1 verificada simbólicamente, 1 respaldada por una fuente.
¶39: dos testimonios.
¶40: dos testimonios. Ecuaciones: 1 verificada simbólicamente, 5 respaldada por una fuente.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

Alan M. Turing — On Computable Numbers, with an Application to the Entscheidungsproblem (1936) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
John von Neumann — First Draft of a Report on the EDVAC (1945) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
C Programming Language Reference (ISO/IEC 9899 C17/C23 — cppreference) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
The Rust Programming Language (Klabnik & Nichols — Official Rust Documentation) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
ECMA-262 — ECMAScript 2024 Language Specification (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
The Go Programming Language Specification & Memory Model (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
RFC 3629 — UTF-8, a transformation format of ISO 10646 (IETF Standard) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
MIT OpenCourseWare 6.006 — Introduction to Algorithms (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
The Java Language Specification & JVM Specification (Java SE 21 LTS) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
The Rust Reference — Memory Model, Type System, Lifetimes, and Unsafe Rust (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The TypeScript Handbook — Structural Typing, Narrowing, and Type Manipulation (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Python Language Reference & Standard Library Documentation (Python 3.12–3.14) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-antecedentes-y-computabilidad scientific
REV-0002: cap02-primeros-ordenadores-y-compiladores scientific
REV-0003: cap03-paradigmas-sistemas-y-era-moderna scientific
REV-0004: cap04-representacion-binaria-ieee754-utf8 scientific
REV-0005: cap05-logica-algebra-booleana-matematica-discreta scientific
REV-0006: cap06-arquitectura-cpu-memoria-compilacion scientific
REV-0007: cap07-sistemas-de-tipos-control-ambito-errores scientific
REV-0008: cap08-los-doce-paradigmas-de-programacion scientific

Apéndice E

Plan de estudio

PARTE I. Historia de la Informática y la Programación

✓Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

- Dominar Antecedentes matemáticos, máquinas analíticas, álgebra de Boole, Turing y el cálculo lambda

2 conceptos planificados.

✓Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

- Dominar Primeros ordenadores electrónicos, ensamblador, compiladores, Grace Hopper, FORTRAN, Lisp, COBOL, ALGOL, C y Unix

2 conceptos planificados.

✓De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

- Dominar De Smalltalk, C++, Java, Python, JS y C# a Git, Linux, Go, Rust, Kotlin, Swift, Cloud, Wasm e IA

2 conceptos planificados.

PARTE II. Fundamentos Universales de Programación y Matemáticas Discretas

✓Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

- Dominar Representación binaria, complemento a dos, coma flotante IEEE 754, Unicode y UTF-8

2 conceptos planificados.

✓Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

- Dominar Álgebra booleana, lógica proposicional y de predicados, conjuntos, inducción, combinatoria y grafos

2 conceptos planificados.

✓Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

- Dominar Arquitectura de computadores, CPU, registros, memoria, E/S, interpretación, compilación AOT y JIT

2 conceptos planificados.

✓Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

- Dominar Sistemas de tipos, variables, ámbito léxico, tiempo de vida, clausuras, modularidad y gestión de errores

2 conceptos planificados.

✓Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

- Dominar Los doce paradigmas de programación: fundamentos, compromisos, límites y criterios de selección

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering